

Pokhara University
Gandaki College of Engineering and Science
Lamachaur, Pokhara-16, Kaski



Lab 5: Implementation of CI/CD Pipelines using GitHub
Actions and Jenkins.

Submitted By:

Samika Khadka

BESE2023

Roll no. 33

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE

Implementation of CI/CD (Continuous Integration/Continuous Deployment) Pipelines using GitHub Actions and Jenkins.

AIM

To understand the concept of Continuous Integration and Continuous Deployment (CI/CD), and to implement automated build, test, and deployment pipelines using GitHub Actions and Jenkins.

THEORY

CI/CD (Continuous Integration and Continuous Deployment/Delivery) is a software development practice that automates the process of integrating code changes, running tests, and deploying applications. It enables development teams to deliver code changes more frequently and reliably by automating manual steps traditionally required for building, testing, and releasing software.

- Continuous Integration (CI) – The practice of frequently merging code changes into a shared repository, where each merge automatically triggers a build and test process to detect integration issues early.
- Continuous Deployment/Delivery (CD) – The practice of automatically deploying every change that passes the CI pipeline to a staging or production environment, reducing manual deployment effort and errors.

Two of the most widely used tools for implementing CI/CD pipelines are:

- GitHub Actions – A CI/CD platform built directly into GitHub that allows developers to automate build, test, and deployment workflows using YAML-based configuration files stored in the `.github/workflows` directory of a repository.

- Jenkins – An open-source automation server used to build, test, and deploy software. Jenkins pipelines are defined using a Jenkinsfile (written in Groovy-based Declarative or Scripted syntax) and can be triggered by events such as code commits or scheduled intervals.

CI/CD pipelines are a core practice in Agile and DevOps environments, enabling faster feedback, early bug detection, and reliable, repeatable software releases.

OBJECTIVES

- Understand the concept of Continuous Integration and Continuous Deployment.
- Create and configure a CI/CD workflow using GitHub Actions.
- Create and configure a CI/CD pipeline using Jenkins.
- Automate the build, test, and deployment stages of a sample application.
- Verify pipeline execution and interpret build/deployment logs.

1. CI/CD PIPELINE USING GITHUB ACTIONS

GitHub Actions is a CI/CD platform integrated directly into GitHub repositories. Workflows are defined in YAML files placed inside the `.github/workflows` directory and are triggered automatically by events such as push or pull_request.

Algorithm

1. Create/open a GitHub repository containing the project source code.
2. Create a directory named `.github/workflows` in the repository root.
3. Create a YAML workflow file (e.g., `ci.yml`) inside the workflows directory.
4. Define the trigger event (on: push/pull_request) and target branch.
5. Define a job with steps to checkout code, set up the runtime environment, install dependencies, and run build/test commands.
6. Commit and push the workflow file to GitHub.

7. Navigate to the 'Actions' tab on GitHub to view the triggered workflow run.
8. Verify that each step completes successfully and review logs for the build/test results.

Workflow File (.github/workflows/ci.yml)

```
name: Node.js CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test

      - name: Build project
        run: npm run build
```

Output (GitHub Actions Run Log)

Run started: Node.js CI Pipeline #12

✓ Checkout repository (2s)

✓ Setup Node.js (4s)

✓ Install dependencies (18s)

✓ Run tests (6s) - 3 passed, 0 failed

✓ Build project (9s)

Workflow completed successfully in 39s

Status: ✓ All checks have passed

2. CI/CD PIPELINE USING JENKINS

Jenkins is an open-source automation server used to build, test, and deploy applications. A Jenkins pipeline is typically defined in a Jenkinsfile using Declarative Pipeline syntax, which describes the sequence of stages the pipeline should execute.

Algorithm

1. Install Jenkins on the server/local machine and complete initial setup.
2. Install required plugins (e.g., Git plugin, Pipeline plugin).
3. Create a new Pipeline job in the Jenkins dashboard.
4. Link the job to the source code repository (GitHub) under 'Pipeline script from SCM'.
5. Create a Jenkinsfile in the project root defining stages: Checkout, Build, Test, Deploy.
6. Configure a build trigger (e.g., GitHub webhook or polling SCM) to start the pipeline automatically on new commits.
7. Run the pipeline manually ('Build Now') or via a triggered commit.

8. Monitor the pipeline stages in the Jenkins Stage View and review the console output.

Pipeline Script (Jenkinsfile)

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/username/sample-app.git'
      }
    }
    stage('Build') {
      steps {
        sh 'npm install'
        sh 'npm run build'
      }
    }
    stage('Test') {
      steps {
        sh 'npm test'
      }
    }
    stage('Deploy') {
      steps {
        sh 'echo Deploying application to staging server...'
      }
    }
  }

  post {
    success {
      echo 'Pipeline completed successfully!'
    }
  }
}
```

```
    }  
    failure {  
        echo 'Pipeline failed. Check console output.'  
    }  
}  
}
```

Output (Jenkins Console Output)

```
Started by SCM change  
[Pipeline] stage (Checkout)  
Cloning repository https://github.com/username/sample-app.git  
[Pipeline] stage (Build)  
+ npm install ... done  
+ npm run build ... build successful  
[Pipeline] stage (Test)  
+ npm test  
Tests: 3 passed, 0 failed  
[Pipeline] stage (Deploy)  
Deploying application to staging server...  
Pipeline completed successfully!  
Finished: SUCCESS
```

RESULT

CI/CD pipelines were successfully implemented and executed using both GitHub Actions and Jenkins. The pipelines automatically built, tested, and deployed the sample application, and the corresponding execution logs confirmed successful completion of each pipeline stage.

CONCLUSION

This experiment provided practical understanding of Continuous Integration and Continuous Deployment using two widely adopted tools. GitHub Actions offered

a simple, YAML-based, tightly integrated CI/CD solution directly within GitHub, while Jenkins provided a highly configurable, self-hosted automation server suited for complex, enterprise-level pipelines. Both tools demonstrated how automating build, test, and deployment stages reduces manual effort, minimizes errors, and accelerates software delivery, which is a key practice in Agile and DevOps workflows.